

Práctica: Docker y Docker Compose

Sistemas de Gestión Empresarial

Índice

1. Introducción	3
2. Descarga de Imágenes de Docker	3
2.1. Explicación Teórica	3
2.2. Comandos Guiados	3
3. Creación y Gestión de Contenedores	3
3.1. Explicación Teórica	3
3.2. Comandos Guiados	3
3.3. Administración de Contenedores: Logs, Consola, Detención y Borrado . . .	4
4. Gestión de Volúmenes y Persistencia de Datos	5
4.1. Explicación Teórica	5
4.2. Comandos Guiados	5
5. Redes en Docker	5
5.1. Explicación Teórica	5
5.2. Comandos Guiados	5
6. Orquestación Completa con Docker Compose	6
6.1. Explicación Teórica	6
6.2. Estructura del Archivo <code>docker-compose.yml</code>	6
6.3. Comandos de Gestión con Docker Compose	7
7. Ejercicios	7
7.1. Imágenes de Docker	7

7.2. Creación y gestión de contenedores	8
7.3. Gestión de volúmenes y persistencia de datos	8
7.4. Redes en Docker	8
7.5. Orquestación completa con docker compose	8

1. Introducción

En esta práctica aprenderemos los conceptos fundamentales de la virtualización basada en contenedores orientada al despliegue de sistemas ERP. Utilizaremos **Docker** para gestionar imágenes, contenedores, volúmenes de persistencia y redes aisladas, finalizando con la orquestación completa de un entorno con **Odoo** y **PostgreSQL** mediante **Docker Compose**.

2. Descarga de Imágenes de Docker

2.1. Explicación Teórica

Una **imagen de Docker** es un paquete ejecutable independiente que incluye todo lo necesario para ejecutar una aplicación: código, entorno de ejecución, bibliotecas y configuraciones.

Para desplegar Odoo y su base de datos PostgreSQL, necesitamos obtener sus respectivas imágenes desde el registro público **Docker Hub**.

2.2. Comandos Guiados

Para buscar y descargar las imágenes oficiales, ejecutamos en la terminal:

```
# Descargar la imagen de Odoo
docker pull odoo:16.0

# Descargar la imagen de PostgreSQL
docker pull postgres:15
```

Para verificar las imágenes descargadas en nuestro sistema:

```
docker images
```

3. Creación y Gestión de Contenedores

3.1. Explicación Teórica

Un **contenedor** es una instancia en ejecución de una imagen. Los contenedores están aislados entre sí y del sistema anfitrión, permitiendo ejecutar entornos limpios y reproducibles.

3.2. Comandos Guiados

Vamos a crear y ejecutar un contenedor de pruebas con la base de datos PostgreSQL:

```
# Crear un contenedor de PostgreSQL llamado 'mi_postgres'
docker run -d \
  --name mi_postgres \
  -e POSTGRES_USER=odoo \
  -e POSTGRES_PASSWORD=odoo_clave \
  -e POSTGRES_DB=postgres \
  postgres:15
```

Para listar los contenedores activos:

```
# Listar contenedores en ejecucion
docker ps

# Listar todos los contenedores (activos e inactivos)
docker ps -a
```

3.3. Administración de Contenedores: Logs, Consola, Detención y Borrado

Para gestionar el ciclo de vida de los contenedores creados, utilizamos los siguientes comandos básicos:

```
# 1. Ver los logs (registros de salida) del contenedor
docker logs mi_postgres

# Seguir los logs en tiempo real
docker logs -f mi_postgres

# 2. Acceder a la consola de comandos (terminal interactiva) dentro del
    ➔ contenedor
docker exec -it mi_postgres bash

# 3. Detener e iniciar contenedores
docker stop mi_postgres
docker start mi_postgres

# 4. Eliminar contenedores
# Nota: Debes detener el contenedor antes de eliminarlo (o usar -f para forzar)
docker stop mi_postgres
docker rm mi_postgres

# Eliminar todos los contenedores detenidos a la vez
docker container prune
```

4. Gestión de Volúmenes y Persistencia de Datos

4.1. Explicación Teórica

Por defecto, los datos dentro de un contenedor se pierden al eliminarlo. Para garantizar la persistencia de los datos de Odoo (adjuntos, módulos) y de la base de datos PostgreSQL, utilizamos **Volúmenes de Docker**.

4.2. Comandos Guiados

Creación de volúmenes explícitos, asociación a contenedores y eliminación:

```
# Crear volúmenes independientes
docker volume create db_data_odoo
docker volume create odoo_web_data

# Listar los volúmenes existentes
docker volume ls

# Crear un contenedor vinculando explícitamente un volumen (-v)
docker run -d \
  --name postgres_persistente \
  -v db_data_odoo:/var/lib/postgresql/data \
  -e POSTGRES_USER=odoo \
  -e POSTGRES_PASSWORD=odoo_clave \
  postgres:15

# Eliminar un volumen específico (el volumen no debe estar en uso por ningún
  ↪ contenedor)
docker volume rm db_data_odoo

# Eliminar todos los volúmenes no utilizados
docker volume prune
```

5. Redes en Docker

5.1. Explicación Teórica

Para que el contenedor de Odoo se comuniquen con el contenedor de la base de datos de manera segura, ambos deben estar conectados a la misma **Red Virtual de Docker**.

5.2. Comandos Guiados

Creación de una red de tipo bridge, interconexión de contenedores y borrado de redes:

```
# Crear la red personalizada
```

```

docker network create red_odoo

# Listar redes existentes
docker network ls

# Ejecutar PostgreSQL dentro de la red
docker run -d \
  --name db_odoo \
  --network red_odoo \
  -v db_data_odoo:/var/lib/postgresql/data \
  -e POSTGRES_USER=odoo \
  -e POSTGRES_PASSWORD=odoo_pass \
  -e POSTGRES_DB=postgres \
  postgres:15

# Ejecutar Odoo dentro de la misma red
docker run -d \
  --name app_odoo \
  --network red_odoo \
  -p 8069:8069 \
  -v odoo_web_data:/var/lib/odoo \
  -e HOST=db_odoo \
  -e USER=odoo \
  -e PASSWORD=odoo_pass \
  odoo:16.0

# Eliminar una red (debe estar libre de contenedores conectados)
docker network rm red_odoo

# Eliminar todas las redes no utilizadas
docker network prune

```

6. Orquestación Completa con Docker Compose

6.1. Explicación Teórica

Docker Compose nos permite definir todos los servicios, redes y volúmenes en un único archivo YAML denominado `docker-compose.yml`, simplificando la gestión de la infraestructura.

6.2. Estructura del Archivo `docker-compose.yml`

```

services:
  web:
    image: odoo:16.0
    container_name: odoo_web
    depends_on:

```

```

    - db
  ports:
    - "8069:8069"
  environment:
    - HOST=db
    - USER=odoo
    - PASSWORD=odoo_secret_password
  volumes:
    - odoo-web-data:/var/lib/odoo

db:
  image: postgres:15
  container_name: odoo_db
  environment:
    - POSTGRES_DB=postgres
    - POSTGRES_USER=odoo
    - POSTGRES_PASSWORD=odoo_secret_password
  volumes:
    - odoo-db-data:/var/lib/postgresql/data

volumes:
  odoo-web-data:
  odoo-db-data:

```

6.3. Comandos de Gestión con Docker Compose

```

# Levantar el entorno en segundo plano
docker compose up -d

# Ver el estado de los servicios
docker compose ps

# Detener y eliminar los contenedores y redes creados por compose
docker compose down

```

7. Ejercicios

7.1. Imágenes de Docker

1. Descarga la versión 15.0 de la imagen de Odoo.
2. Comprueba el espacio ocupado en disco por cada una de las imágenes descargadas.
3. ¿Qué diferencia existe entre ejecutar `docker pull odoo` y `docker pull odoo:16.0`?

7.2. Creación y gestión de contenedores

1. Detén el contenedor `mi_postgres` y vuélvelo a iniciar.
2. Consulta los logs del contenedor `mi_postgres` para comprobar que se inició correctamente.
3. Entra por la consola interactiva (`bash`) al contenedor `mi_postgres` y ejecuta el comando `psql -U odoo -d postgres` para verificar la base de datos.
4. Ejecuta un contenedor temporal de Odoo mapeando el puerto 8069 del contenedor al puerto 8069 de tu máquina local.
5. Accede a `http://localhost:8069` en tu navegador y comprueba qué sucede al no estar conectado a PostgreSQL.
6. Detén y elimina todos los contenedores creados en esta sección.

7.3. Gestión de volúmenes y persistencia de datos

1. Crea un contenedor de PostgreSQL utilizando el volumen `db_data_odoo` montado en la ruta interna `/var/lib/postgresql/data`.
2. Averigua la ruta física del sistema anfitrión donde Docker almacena los volúmenes usando `docker volume inspect`.
3. Detén el contenedor, elimínalo y posteriormente borra el volumen `db_data_odoo` con `docker volume rm`.

7.4. Redes en Docker

1. Inspecciona la red mediante `docker network inspect red_odoo` para comprobar las direcciones IP asignadas.
2. Entra por consola interactiva al contenedor de Odoo e comprueba la conectividad hacia el contenedor de base de datos haciendo `ping db_odoo`.
3. Detén y elimina ambos contenedores, y elimina la red creada usando `docker network rm red_odoo`.

7.5. Orquestación completa con `docker compose`

1. **Despliegue:** Guarda el archivo `docker-compose.yml`, levanta el entorno y crea una base de datos de prueba desde el navegador web.
2. **Comprobación de Persistencia:** Ejecuta `docker compose down`, vuelve a levantar los servicios con `docker compose up -d` y verifica que la información creada en Odoo siga intacta.

3. **Personalización:** Modifica el archivo `docker-compose.yml` para descargar odoo en su version 19 y vincula una carpeta local llamada `./addons` con la ruta interna de Odoo `/mnt/extra-addons`.
4. Capturas de pantalla a incluir en la entrega:
 - Salida de los comandos `docker images` y `docker ps`.
 - Archivo `docker-compose.yml` modificado con el volumen local de addons.
 - Interfaz web de Odoo funcionando correctamente.